

עבודה על פרוייקט מסכם

מודול פייתון

TomerBu

דגשים לפרוייקט

מסמך דרישות לפרוייקט

התקנה חד-פעמית:

```
pip install django
```

מתקין את django
גלובלית על המחשב

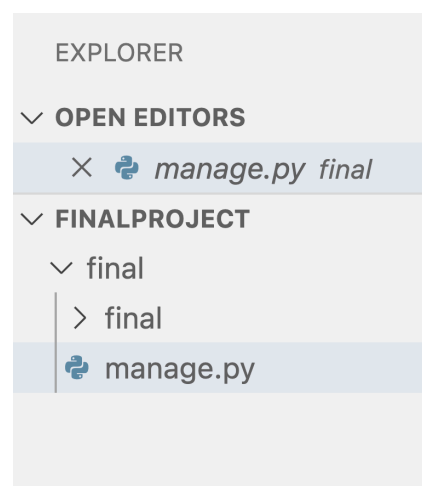
פרוייקט חדש:

צרו תיקיה חדשה:

ופתחו את התיקיה בעזרת vscode

נריץ בטרמינל פקודה ליצירת פרוייקט חדש:

```
django-admin startproject final
```



יצירת VENV

בדומה לnode_modules (אנחנו רוצים להתקין את כל הספריות בתיקה בתוך הפרוייקט)

```
python -m venv .venv
```

```
.venv/scripts/activate
```

כעת כל `pip install`

יתקין את הספרייה לתיקה בתוך הפרוייקט

טוב כשאנו רוצים לשלוח את הפרוייקט ולעבוד בצוות

התקנה של ספריות בפרוייקט:

התקנה מומלצת |mysql

<https://dev.mysql.com/downloads/installer/>

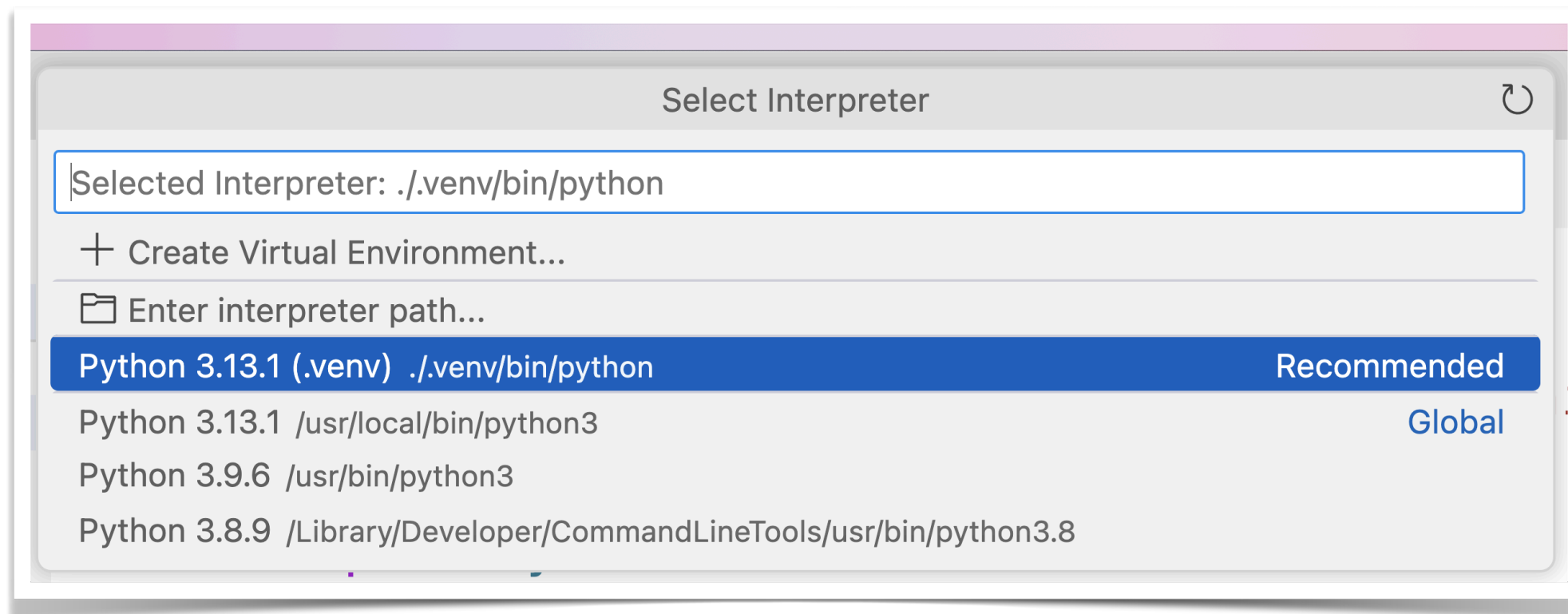
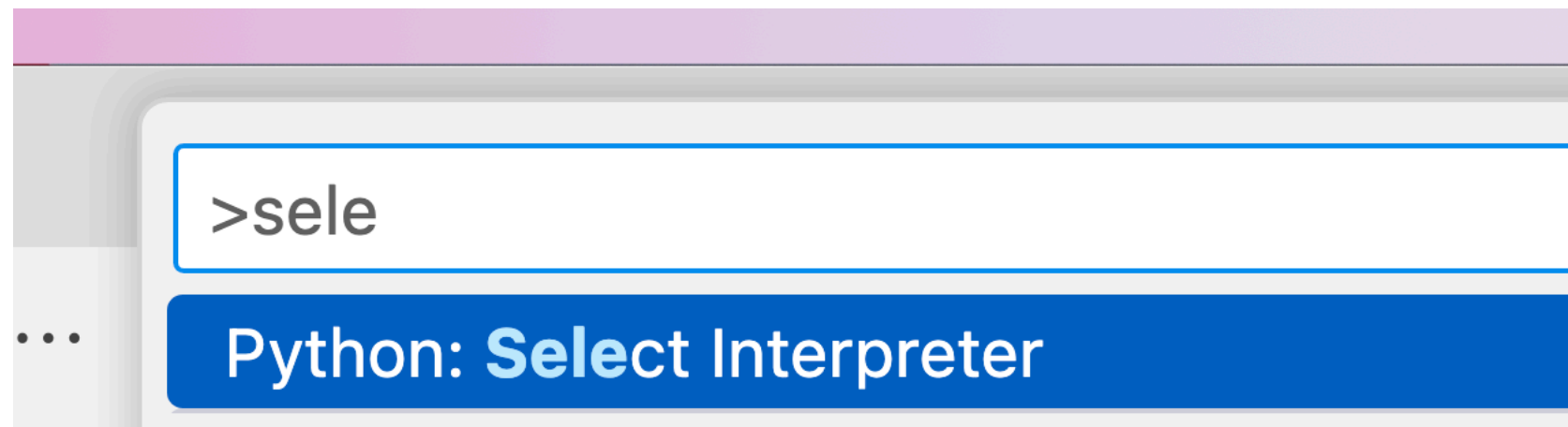
```
pip install django djangorestframework mysqlclient
```

DRF יודעת לעבוד גם עם Postgresql

<https://www.postgresql.org/download/windows/>

השלמה אוטומטית בVSCode

ctrl+shift+p



סיכום עד כה:

התקנו mysql עבור חלונות

- `pip install django`
- `django-admin startproject final`
- `python manage.py startapp api`
- `python -m venv .venv`
- `.venv/scripts/activate`
- `pip install django djangorestframework`
- `python manage.py startapp api`
- `ctrl+shift+p` select interpreter

סיכום עד כה:

התקנו mysql עבור חלונות

- `pip install django`
- `django-admin startproject final`
- `python manage.py startapp api`
- `python -m venv .venv`
- `.venv/scripts/activate`
- `pip install django djangorestframework`
- `python manage.py startapp api`
- `ctrl+shift+p` select interpreter

מודול חדש בפרוייקט:

- נרצה ליצור תיקיה עבור הAPI

- (חלוקת עבודה לתיקיות)

נריץ בטרמינל את הפקודה הבאה:

python manage.py startapp api

↑
פקודה
ליצירת
מודול

↑
שם התיקיה

הגדרות settings.py עבור המודולים שהתקנו:

- (1) המודול החדש שלנו api
- (2) המודול djangorestframework

settings.py:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'api',  
    'rest_framework',  
]
```

שילבי עבודה בבניית API:

חשוב להגדיר היטב את המודלים

1) model + migration + admin

2) Json Serializers ממיר אובייקט פייתון לג'ייסון

3) Views - CRUD מימוש כל פעולות ה

4) urls קביעת נתיבים לפונקציות שהגדרנו בשלב הקודם

אחרי שסיימנו - יש לנו API

המודלים:

User

UserProfile

Tags

Post

Comment

PostUserLikes

המודלים:

Post

סטטוס:

draft
published
archived

המודלים:

article

תודה תומר

reply_to:null

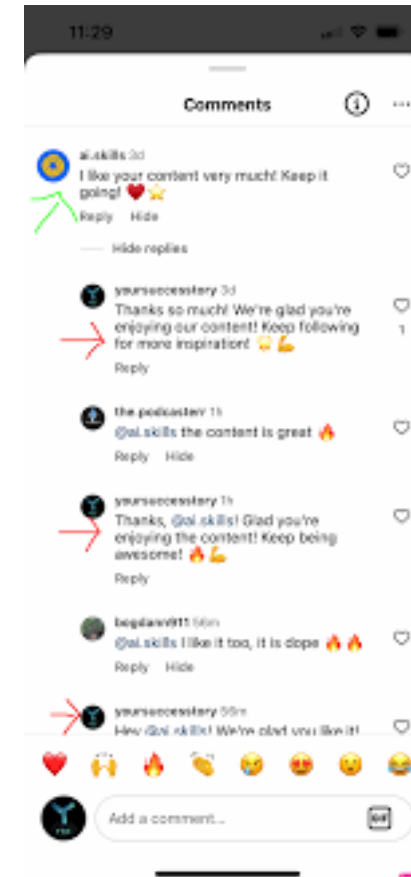
איזה כיף
שהגבת!

reply_to:1

comment => comment

אחד לאחד

מתגובה לתגובה



המודלים:

User

UserProfile

למה צריך משתמש: לזהות משתמש, לדעת מי כתב פוסט, מי כתב תגובה!

למה צריך 2 מחלקות למשתמש?

בעת הרשמה נבקש מהמשתמש שם משתמש אימייל וסיסמא

בהמשך לאט לאט - נדלה ממנו עוד פרטים

ואת זה נשמור במידע הנוסף

מודל מובנה בdjango

User

מודל משלנו עם קשרי גומלין לuser
אחד-לאחד

UserProfile

Please complete this online form to submit your request.

Title:	Dr	
First Name:		*
Last Name:		*
Company:		*
Address 1:		
Address 2:		
City:		*
County:		
Post Code:		
Telephone:		*
E-Mail:		*
Fax:		
What are you?	Customer	
Department to contact:	Sales	
Contact method:	Email	
Your request:		

טופס הרשמה בעייתי

המודלים:

Tags

נושאים לכתבות:

לדוגמא typescript

javascript

css

html

react

כתבה בנושא:

css וגם react

לכתבה אחת יש הרבה נושאים:

קשר גומלין אחד לרבים

המודלים:

לייקים לכתבות:

PostUserLikes

משתמש יכול לעשות לייק להרבה כתבות

כתבה יכולה לקבל לייק מהרבה משתמשים

רבים לרבים
many to many

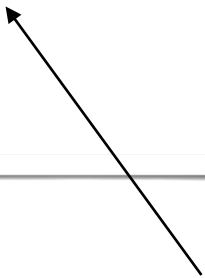
המודלים:

api/models.py

```
from django.db import models

class Tag(models.Model):
    # props
    name = models.CharField(unique=True, max_length=32)

    # str representation for the admin panel
    def __str__(self):
        return self.name
```



נראה בהמשך בממשק הניהול
admin panel

api/models.py

```
from django.contrib.auth.models import User

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, unique=True)
    bio = models.TextField(blank=True, max_length=1000)
    birthdate = models.DateField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

המודלים:

api/models.py

```
# Post status
STATUS_CHOICES = [
    ('draft', 'Draft'),
    ('published', 'Published'),
    ('archived', 'Archived')
]

class Post(models.Model):
    author = models.ForeignKey(
        UserProfile, on_delete=models.CASCADE)

    title = models.CharField(max_length=100, unique=True, validators=[
        MinLengthValidator(5),
        RegexValidator(regex='^[a-zA-Z].*$')
    ])
    text = models.TextField(validators=[MinLengthValidator(5)])
    tags = models.ManyToManyField(Tag, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    status = models.CharField(choices=STATUS_CHOICES, default='draft')

    def __str__(self):
        return f'{self.title} by {self.author.user.username}'
```

אחד לרבים
foreign key
(לא unique)

המודלים:

api/models.py

```
class Comment(models.Model):
    author = models.ForeignKey(
        UserProfile, on_delete=models.CASCADE
    )
    post = models.ForeignKey(
        Post, on_delete=models.CASCADE
    )
    text = models.TextField(max_length=200, validators=[MinLengthValidator(1)])

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    reply_to = models.ForeignKey('self',
        default=None,
        null=True,
        blank=True,
        on_delete=models.CASCADE
    )

    def __str__(self):
        return f'{self.text} by {self.author.user.username}'
```

אחד לרבים
foreign key
(לא unique)

המודלים:

api/models.py

```
# Like options:
LIKE_CHOICES = [
    ('like', 'Like'),
    ('dislike', 'Dislike'),
]

class PostUserLikes(models.Model):
    # props:
    user = models.ForeignKey(UserProfile, on_delete=models.CASCADE)
    post = models.ForeignKey(Post, on_delete=models.CASCADE)
    like_type = models.CharField(choices = LIKE_CHOICES, default='like')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ['user', 'post']

    def __str__(self):
        return f'{self.user.user.username} {self.like_type}d {self.post.title}'
```

database migrations:

כרגע הגדרנו רק מחלקות/מודלים
נרצה לבנות מזה דטה-בייס

כותב קוד של מיגרציות:

כלומר קוד שמתרגם את המודלים שלנו לsql

```
python manage.py makemigrations api
```

מבצע בפועל את המיגרציה -

כלומר מריץ את הsql מול הדטה-בייס

```
python manage.py migrate
```


ממשק ניהול admin panel

python manage.py createsuperuser

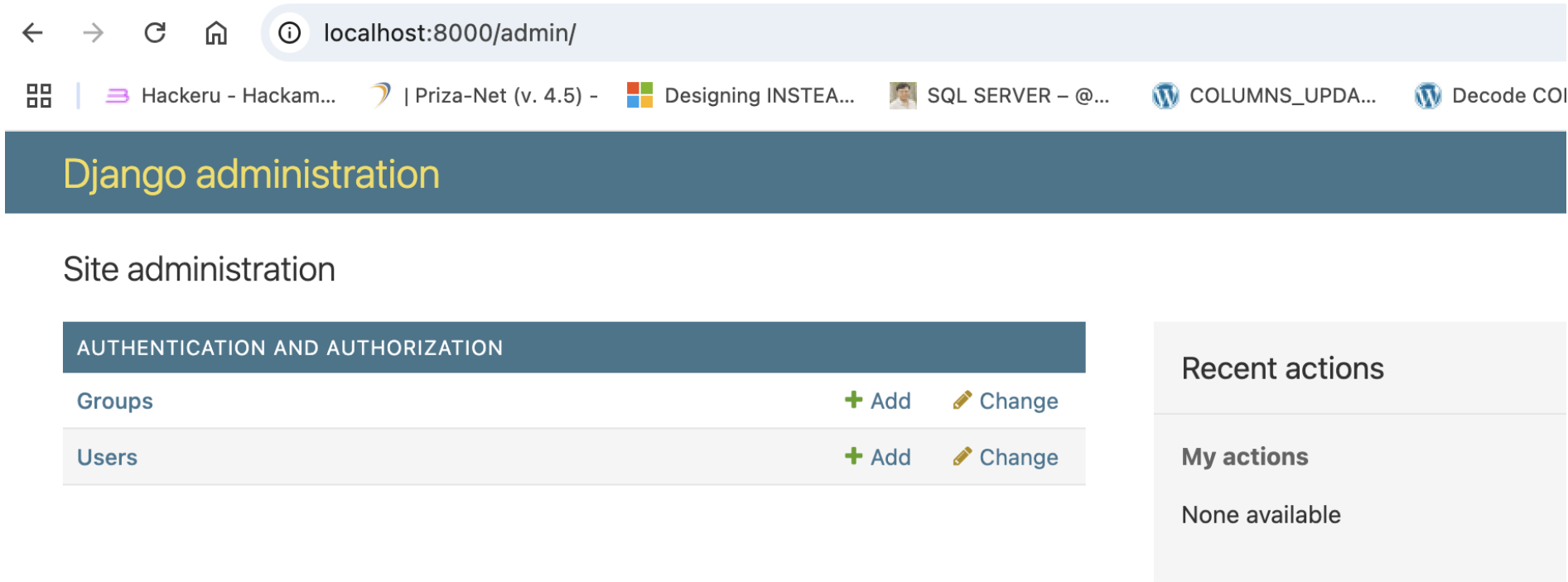
1

python manage.py runserver

2

localhost:8000/admin

3



הוספת המודלים שלנו לממשק הניהול:

localhost:8000/admin

```
from django.contrib import admin

from .models import Comment, UserProfile, Post, Tag, PostUserLikes

# Register your models here.
admin.site.register([Comment, UserProfile, Post, Tag, PostUserLikes])
```

Serializers

api/serializers.py

```
from django.contrib.auth.models import User
from rest_framework.serializers import ModelSerializer
from api.models import Comment, Post, PostUserLikes, Tag, UserProfile

class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = "__all__"

class CommentSerializer(ModelSerializer):
    class Meta:
        model = Comment
        fields = "__all__"

class UserProfileSerializer(ModelSerializer):
    class Meta:
        model = UserProfile
        fields = "__all__"

class PostSerializer(ModelSerializer):
    class Meta:
        model = Post
        fields = "__all__"

class TagSerializer(ModelSerializer):
    class Meta:
        model = Tag
        fields = "__all__"

class PostUserLikeSerializer(ModelSerializer):
    class Meta:
        model = PostUserLikes
        fields = "__all__"
        # fields = ['text', 'id']
```

בהמשך נשפר
(כלומר נתאים את הSerializers שלנו)

Views:

```
from rest_framework.viewsets import ModelViewSet

from .serializers import (UserSerializer, PostSerializer,
                           UserProfileSerializer, CommentSerializer,
                           TagSerializer, PostUserLikeSerializer)

from .models import Tag, UserProfile, Post, Comment, PostUserLikes

# פעולות CRUD
# GET /api/tags
# GET /api/tags/1
# POST /api/tags
# DELETE /api/tags/1
# PUT /api/tags/1
# PATCH /api/tags/1
class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all() # SELECT * FROM Tags
    serializer_class = TagSerializer # converts objects to JSON
```

מימוש של כל פעולות ה-CRUD

Views:

api/views.py

```
from rest_framework.viewsets import ModelViewSet
from django.contrib.auth.models import User
from .serializers import (UserSerializer, PostSerializer,
                          UserProfileSerializer, CommentSerializer,
                          TagSerializer, PostUserLikesSerializer)

from .models import Tag, UserProfile, Post, Comment, PostUserLikes

class UserViewSet(ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer

class PostUserLikesViewSet(ModelViewSet):
    queryset = PostUserLikes.objects.all()
    serializer_class = PostUserLikesSerializer

class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer

class UserProfileViewSet(ModelViewSet):
    queryset = UserProfile.objects.all()
    serializer_class = UserProfileSerializer

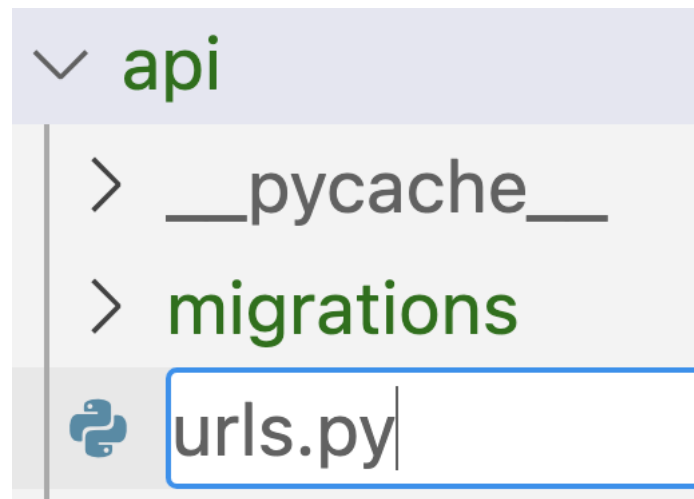
class CommentViewSet(ModelViewSet):
    queryset = Comment.objects.all()
    serializer_class = CommentSerializer
```

מימוש שלם של כל פעולות הCRUD

urls:

api/urls.py

ניצור קובץ חדש בשם urls.py בתיקיית api:



urls:

api/urls.py

▼ api

> __pycache__

> migrations

🔗 __init__.py

🔗 admin.py

🔗 apps.py

🔗 models.py

🔗 serializers.py

🔗 tests.py

🔗 urls.py

🔗 views.py

```
from rest_framework.routers import DefaultRouter

from .views import (CommentViewSet, PostViewSet, PostUserLikesViewSet,
                    UserProfileViewSet, UserViewSet, TagViewSet)

router = DefaultRouter()

router.register('tags', TagViewSet, basename='tags')
router.register('comments', CommentViewSet, basename='comments')
router.register('posts', PostViewSet, basename='posts')
router.register('users', UserViewSet, basename='users')
router.register('userprofiles', UserProfileViewSet, basename='userprofiles')
router.register('likes', PostUserLikesViewSet, basename='likes')

urlpatterns = router.urls
```

#optional: we can add more patterns here:

```
urlpatterns += [
```

] זה מיותר כרגע - אבל מאפשר להוסיף פונקציה בהמשך שאין לה ראוטר

פה אנו מגדירים נתיבים כמו tags/
כדי לקבל תגובה

urls:

final/urls.py

▼ FINAL

> .venv

> .vscode

> api

▼ final

> __pycache__

 __init__.py

 asgi.py

 settings.py

 urls.py

 wsgi.py

≡ db.sqlite3

 manage.py

final/urls.py

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

```
# api/    we include all the api module endpoints
```


localhost:8000/api/

Hackam... | Priza-Net (v. 4.5) - Designing INSTEA... SQL SERVER - @... COLUMNS_UPDA... Decode COLUMN... Using Columns_U... Introduction to SQ... stanf Toolbar Animation... Hanc

Api Root

Api Root

The default basic root view for DefaultRouter

GET /api/

HTTP 200 OK

Allow: GET, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "tags": "http://localhost:8000/api/tags/",
  "comments": "http://localhost:8000/api/comments/",
  "posts": "http://localhost:8000/api/posts/",
  "users": "http://localhost:8000/api/users/",
  "userprofiles": "http://localhost:8000/api/userprofiles/",
  "likes": "http://localhost:8000/api/likes/"
}
```